

QuizzApp: System Architecture & API Specifications

A Technical Engineering Reference for Developers & AI RAG Systems

Document Type:	System Architecture & API Specs
Target Audience:	Software Engineers & AI RAG Systems
Backend Stack:	FastAPI, SQLAlchemy, PostgreSQL/SQLite, JWT, Bcrypt
Frontend Stack:	React, TypeScript, Tailwind CSS v3, Vite
Specification Version:	v1.2 / English

1. Technology Stack & Topology

QuizzApp runs a decoupled client-server architecture. The frontend is compiled as a React Single Page Application (SPA) utilizing Vite as the bundler and TypeScript to enforce strict typing. The styling system is designed with utility classes from Tailwind CSS v3. The backend is managed by FastAPI running on Uvicorn ASGI server. The relational database layer is modeled via SQLAlchemy ORM with database migrations handled by Alembic. Authentication uses JSON Web Tokens (JWT) for secure session state, and passwords are encrypted using Bcrypt.

2. Database Schema (SQLAlchemy Models)

The relational database schema is structured into the following SQLAlchemy models mapping to core tables:

2.1 User & Settings Models

- **User Table:** Manages accounts. Columns: id (PK, Int), username (Str, Unique), email (Str, Unique), full_name (Str), password (Str/Hashed), is_active (Bool), role (Str: STUDENT/TEACHER/SUPER_ADMIN), status (Str: ACTIVE/BLOCKED/UNVERIFIED), avatar_url (Str, Nullable), registration_type (Str, Default: 'LOCAL'), email_verified (Bool, Default: False), created_at (DateTime).
- **UserSetting Table:** Stores user settings. Columns: id (PK, Int), user_id (FK to users.id, cascade delete), notification_email (Str, Nullable), global_email (Bool, True), global_in_app (Bool, True), system_announcements (Bool, True), exam_assignments (Bool, True), exam_reminders (Bool, True), results_published (Bool, True), room_invitations (Bool, True).
- **RefreshToken Table:** Tracks active sessions. Columns: id (PK, Int), user_id (FK to users.id), token (Str, Unique), expires_at (DateTime), revoked (Bool, False), created_at (DateTime).

2.2 Group & Member Models

- **Group Table:** Manages classroom groups. Columns: id (PK, Int), owner_id (FK to users.id), name (Str), description (Text), group_code (Str, Unique), status (Str, Default: 'OPEN' [OPEN/CLOSED]), icon (Str, Default: 'GraduationCap'), created_at (DateTime).
- **GroupMember Table:** Maps students to groups. Columns: id (PK, Int), group_id (FK to groups.id, cascade), user_id (FK to users.id, cascade), role_in_group (Str, Default: 'STUDENT'), status (Str, Default: 'APPROVED' [PENDING/INVITED/APPROVED/BLOCKED]), invited_by (FK to users.id, Nullable), requested_at (DateTime), joined_at (DateTime).

2.3 Quiz & Question Models

- **Quiz Table:** Assessment metadata. Columns: id (PK, Int), title (Str), description (Text), creator_id (FK to users.id), is_public (Bool, False), category (Str, Default: 'Science'), duration (Int, Default: 60), icon (Str), created_at (DateTime).
- **Question Table:** Question slides. Columns: id (PK, Int), quiz_id (FK to quizzes.id, cascade), question_text (Text), question_type (Str: MCQ/TRUE_FALSE/SHORT_ANSWER), time_limit (Int, Default: 60), points (Int, Default: 10), audio_url (Str, Nullable).
- **QuestionOption Table:** Answer options. Columns: id (PK, Int), question_id (FK to questions.id, cascade), option_text (Str), is_correct (Bool, False), audio_url (Str, Nullable).

2.4 Live Room & Play Models

- **Room Table:** Sync multiplayer room. Columns: id (PK, Int), quiz_id (FK to quizzes.id), host_id (FK to users.id), pin (Str, Unique), status (Str [LOBBY/PLAYING/FINISHED]), created_at (DateTime).
- **Participant Table:** Player in a live room. Columns: id (PK, Int), room_id (FK to rooms.id), name (Str), score (Int, Default: 0), user_id (FK to users.id, Nullable), status (Str).

- **ParticipantAnswer Table:** Realtime responses. Columns: id (PK, Int), participant_id (FK to participants.id, cascade), question_id (FK to questions.id), option_id (FK to question_options.id, Nullable), points_awarded (Int), response_time (Float).

2.5 Official Exam Models

- **Exam Table:** Assigned exams. Columns: id (PK, Int), quiz_id (FK to quizzes.id), group_id (FK to groups.id), creator_id (FK to users.id), title (Str), due_date (DateTime), duration (Int), status (Str [PENDING/ACTIVE/CLOSED]), created_at (DateTime).
- **ExamAssignee Table:** Assigned students. Columns: id (PK, Int), exam_id (FK to exams.id, cascade), user_id (FK to users.id, cascade), status (Str [ASSIGNED/STARTED/SUBMITTED]), score (Float, Nullable), started_at (DateTime, Nullable), submitted_at (DateTime, Nullable).
- **ExamAnswer Table:** Submitted responses. Columns: id (PK, Int), assignee_id (FK to exam_assignees.id, cascade), question_id (FK to questions.id), selected_option_id (FK to question_options.id, Nullable), short_answer_text (Str, Nullable), score (Float).

2.6 Notifications & Achievements Models

- **Notification Table:** System alerts. Columns: id (PK, Int), sender_id (FK to users.id), user_id (FK to users.id), target_type (Str, Default: 'PERSONAL'), target_group_id (FK to groups.id), title (Str), content (Text), type (Str), is_read (Bool, False), action_url (Str), created_at (DateTime).
- **Badge Table:** Badges list. Columns: id (PK, Int), name (Str), description (Text), criteria (Str), image_url (Str).
- **UserBadge Table:** Awarded badges. Columns: id (PK, Int), user_id (FK to users.id), badge_id (FK to badges.id), awarded_at (DateTime).

3. REST API Endpoint Catalog (v1)

The tables below detail all standard HTTP REST endpoints available on the FastAPI backend:

HTTP Method & Path	Auth Scope	Functional Description
POST /auth/login	Public	Authenticate using username/email and password. Returns JWT access_token and refresh_token.
POST /auth/refresh-token	Public	Submit a refresh_token to obtain a new short-lived access_token.
POST /auth/logout	Public	Invalidates and revokes the submitted refresh_token.
POST /auth/register	Public	Register new account. Creates user as UNVERIFIED, triggers verification email via BackgroundTask.
POST /auth/verify-email	Public	Verify account using email verification token. Activates user status to ACTIVE.
POST /auth/forgot-password	Public	Trigger password recovery email containing JWT token (expires in 15 mins).
POST /auth/reset-password	Public	Reset password using recovery token. Updates password, invalidates active refresh tokens.
POST /auth/change-password	Active User	Update password for current active user. Requires current password validation.
GET /auth/me	Active User	Returns details of the currently authenticated user.
GET /users/	Admin	Retrieve paginated list of all users in the system.
POST /users/	Admin	Admin-level creation of new users.
GET /users/{user_id}	Owner/Admin	Retrieve profile information of specific user.
PUT /users/{user_id}	Owner/Admin	Update user details. Regular users are forbidden from changing roles, status, or verification state.
DELETE /users/{user_id}	Admin	Permanently remove a user from the system.
GET /users/me/settings	Active User	Get notification settings. Auto-generates defaults if missing.
PUT /users/me/settings	Active User	Update notification toggles. Excludes direct modification of notification_email.
POST /users/me/notification-email/request	Active User	Request email update. Generates OTP and sends to new address.
POST /auth/verify-notification-email	Active User	Submit verification token to commit and update notification email address.
GET /groups/	Active User	Retrieve groups owned by the current user.
POST /groups/	Active User	Create a study group. Generates unique join code.

GET /groups/invitations	Active User	Retrieve group invitations for current user.
POST /groups/join-request	Active User	Join group using code. Blocks request if group status is LOCKED.
POST /groups/{group_id}/invite	Owner/Admin	Invite user by email. Auto-approves if user has pending request.
POST /groups/{group_id}/accept-invite	Active User	Accept a group invitation and set status to APPROVED.
POST /groups/{group_id}/decline-invite	Active User	Decline group invitation, removing membership record.

4. RAG Chatbot Integration Guidelines

To index and query these documents in a Retrieval-Augmented Generation (RAG) system for the virtual assistant:

- **Text Extraction:** Extract raw text directly from the PDF output using standard parsers (e.g., PyPDF, PDFPlumber).
- **Text Chunking:** Use a Recursive Character Text Splitter configured with a **chunk size of 800 characters** and a **10% overlap (80 characters)**. This preserves code parameters, API tables, and step sequences.
- **Metadata Tagging:** Attach tags to each chunk, such as doc_type (user_specifications vs technical_architecture) and module (auth, groups, quizzes, gameplay) for precise metadata filtering.
- **Vector Embeddings & Retrieval:** Generate embeddings using standard models. Deploy a hybrid search strategy (Dense Vector + BM25 Sparse Search) to retrieve precise matches for endpoints and workflows.
- **System Prompt:** Instruct the LLM to base its answers strictly on these retrieved documents, reducing hallucinations regarding security, group settings, or API parameters.